

Universidad Tecnológica Nacional
Desarrollo con Tecnologías Blockchain

Trabajo Práctico Individual - Clase 07

Repaso integrador: red Hyperledger Fabric local con CouchDB

Alumno: Lucas Chas

Legajo: 95346

Fecha: 09/10/2026

Versiones: Hyperledger Fabric v2.5.16 – Fabric CA v1.5.22

Índice

Introducción

El objetivo de este trabajo es integrar los contenidos vistos entre las clases 02 y 06 en una única práctica: el despliegue de una red Hyperledger Fabric local sobre Docker, el desarrollo de un chaincode, el uso de una base de datos de estado con soporte para consultas enriquecidas y la verificación de las garantías de confianza que ofrece la plataforma (identidad, endoso, inmutabilidad e historial).

A diferencia de la práctica de la Clase 04, en esta oportunidad la red se levanta con CouchDB como World State, lo que habilita las rich queries sobre el contenido de los activos. El caso de estudio es un préstamo de equipamiento entre dos organizaciones con roles diferenciados: Org1 actúa como custodio de los equipos y es quien solicita el préstamo, mientras que Org2 actúa como autoridad y es la única que puede aprobarlo, rechazarlo o registrar la devolución.

En cada apartado se transcribe el comando ejecutado, se adjunta la captura de pantalla correspondiente y se describe el comportamiento observado en el sistema.

1. Versiones de Fabric (marco teórico)

1.1. Línea LTS y última release

La línea de soporte a largo plazo (LTS) de Hyperledger Fabric es la v2.5.x; para este trabajo se fijó la versión v2.5.16 de los peers, el orderer y los binarios, junto con Fabric CA v1.5.22. La última release publicada pertenece a la línea 3.x (v3.1.5). Entre sus aportes principales se encuentran el servicio de ordenamiento tolerante a fallas bizantinas (SmartBFT), el soporte de firmas Ed25519 además de ECDSA y la eliminación de componentes heredados como el system channel y el consenso basado en Kafka. La versión 3.1 incorpora además mejoras de rendimiento para el chaincode, como las operaciones de lectura y escritura por lotes.

Fijar las versiones resulta necesario para que el entorno sea reproducible: los scripts de fabric-samples, las imágenes de Docker y los binarios deben corresponder a la misma versión, y la línea LTS es la más estable y documentada.

1.2. Compatibilidad del chaincode entre v2.5 y v3

El chaincode no depende de la versión del peer. Se ejecuta en un proceso o contenedor independiente y se comunica con el peer mediante el protocolo gRPC de la chaincode shim API, que se mantuvo compatible entre ambas líneas. El contrato desarrollado utiliza únicamente la API pública de fabric-contract-api y fabric-shim (getState, putState, getQueryResult, getHistoryForKey, clientIdentity), y el ciclo de vida del chaincode (empaquetado, instalación, aprobación y commit) es el mismo en v3. Los cambios de la nueva versión se concentran en el servicio de ordenamiento y en la administración de canales, no en la interfaz que usa el contrato.

1.3. Raft frente a SmartBFT

Aspecto	Raft	SmartBFT
Tipo de falla tolerada	Fallas por caída (CFT): nodos que dejan de responder	Fallas bizantinas (BFT): nodos que responden de forma maliciosa o arbitraria

Aspecto	Raft	SmartBFT
Tolerancia	f nodos caídos sobre $2f + 1$	f nodos maliciosos sobre $3f + 1$
Funcionamiento	Un líder replica el log hacia los seguidores	Varias rondas de votación entre los nodos
Costo	Menor cantidad de mensajes y menor latencia	Mayor cantidad de mensajes y mayor latencia

El paso a BFT se justifica cuando los nodos de ordenamiento son operados por organizaciones que no confían entre sí, por ejemplo en un consorcio donde cada miembro aporta su propio ordener y no puede descartarse que alguno altere el orden de las transacciones o censure algunas de ellas. Si todos los orderers están bajo el control de una misma entidad confiable, o el único riesgo a cubrir es la caída de nodos, Raft resulta suficiente y ofrece mejor rendimiento.

2. Entorno de trabajo y versiones

El trabajo se realizó en Windows mediante el Subsistema de Windows para Linux (WSL2) con una distribución Ubuntu. Docker Desktop se encuentra integrado con esa distribución, de modo que los comandos docker ejecutados desde Ubuntu operan sobre el mismo motor de contenedores. Inicialmente se intentó trabajar desde Windows PowerShell, pero los scripts de Fabric están escritos en Bash, por lo que todo el trabajo posterior se realizó desde la terminal de Ubuntu.

Antes de comenzar se instalaron en Ubuntu las herramientas que no estaban disponibles (git, curl, jq y Node.js) y se verificaron las versiones con los siguientes comandos:

```
docker --version
docker compose version
git --version
node --version
npm --version
docker ps
```



```
chasd@Lucas:~$ docker --version
Docker version 29.5.3, build d1c06ef
chasd@Lucas:~$ docker compose version
Docker Compose version v5.1.4
chasd@Lucas:~$ git --version
git version 2.53.0
chasd@Lucas:~$ node --version
v18.20.8
chasd@Lucas:~$ npm --version
10.8.2
chasd@Lucas:~$ docker ps
```

Figura 1. Verificación de versiones dentro de Ubuntu (WSL2).

Herramienta	Versión obtenida
Docker	29.5.3
Docker Compose	v5.1.4
Git	2.53.0
Node.js	v18.20.8
npm	10.8.2
Hyperledger Fabric	v2.5.16 (instalada en la Clase 04)
Fabric CA	v1.5.22

3. Red local con CouchDB

```
cd ~/fabric-labs/fabric-samples/test-network
./network.sh down
./network.sh up createChannel -c inventario -s couchdb
```

[illegible]

```
docker ps --format "table {{.Names}}\t{{.Status}}" | grep -E "NAMES|peer|orderer|couchdb"
```

```

chasd@Lucas: ~/fabric-labs/fabric-samples/test-network
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ docker ps --format "table {{.Names}}\t{{.Status}}" | grep -E "NAMES|peer|orderer|couchdb"
NAMES                                STATUS
peer0.org2.example.com              Up About a minute
peer0.org1.example.com              Up About a minute
couchdb0                             Up About a minute
couchdb1                             Up About a minute
orderer.example.com                  Up About a minute
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$

```

Clase 07 – Lucas Chas – Página 5

reemplazar LevelDB (la opción por defecto, embebida en el peer) por CouchDB, los valores se almacenan como documentos JSON y es posible consultarlos por su contenido.

4. Chaincode: máquina de estados y roles

El chaincode se creó manualmente en Visual Studio Code, conectado a la distribución Ubuntu mediante la extensión WSL. Primero se generó la estructura de carpetas desde la terminal:

```
mkdir -p ~/fabric-labs/inventario-ts/src
code ~/fabric-labs/inventario-ts
```

Dentro de esa carpeta se crearon los cuatro archivos indicados por la guía:

Archivo	Función
package.json	Declara las dependencias (fabric-contract-api, fabric-shim) y los scripts de compilación e inicio.
tsconfig.json	Configura el compilador de TypeScript, habilita los decoradores y define dist/ como salida.
src/index.ts	Punto de entrada: exporta el contrato que Fabric debe exponer.
src/inventario.ts	Contrato InventarioContract con el activo Equipo, la máquina de estados y las consultas.

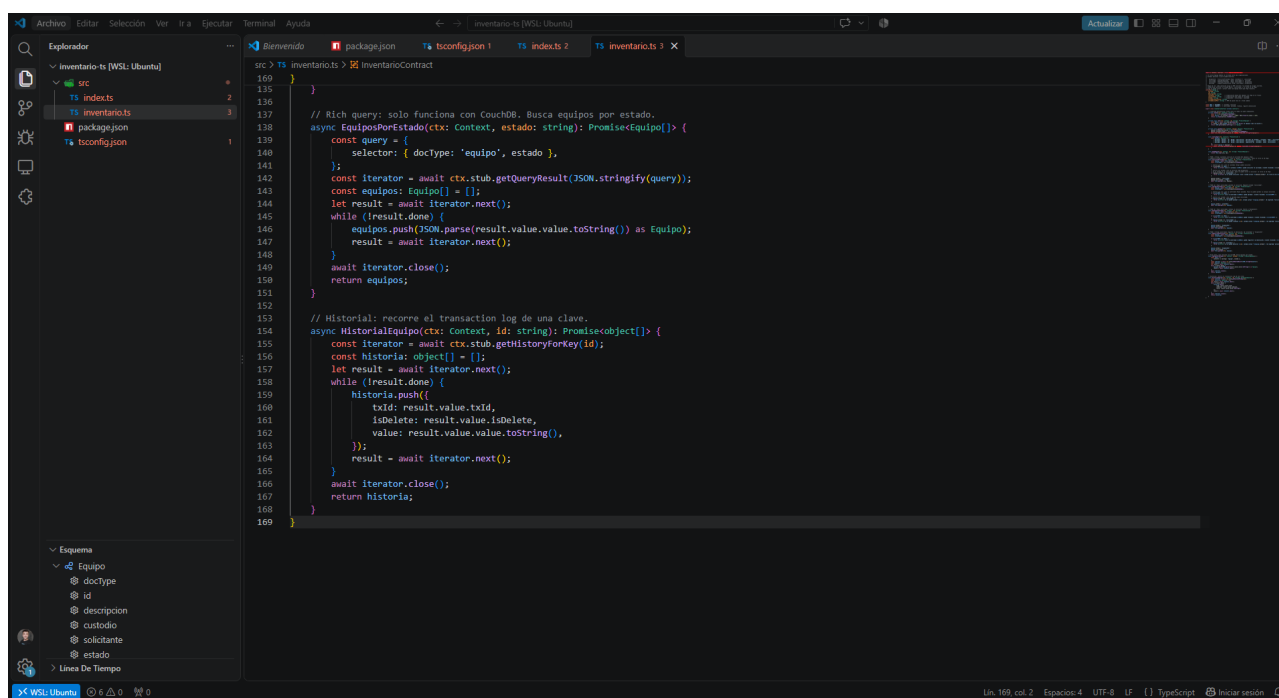


Figura 4. Estructura del chaincode en VS Code (WSL: Ubuntu) con el archivo inventario.ts abierto.

El contrato modela el ciclo de vida de un equipo con tres estados posibles. Cada transición está habilitada únicamente para una organización y desde un estado de origen determinado:

Función	Organización	Estado de origen	Estado resultante
SolicitarPrestamo	Org1 (custodio)	disponible	solicitado
AprobarPrestamo	Org2 (autoridad)	solicitado	prestado

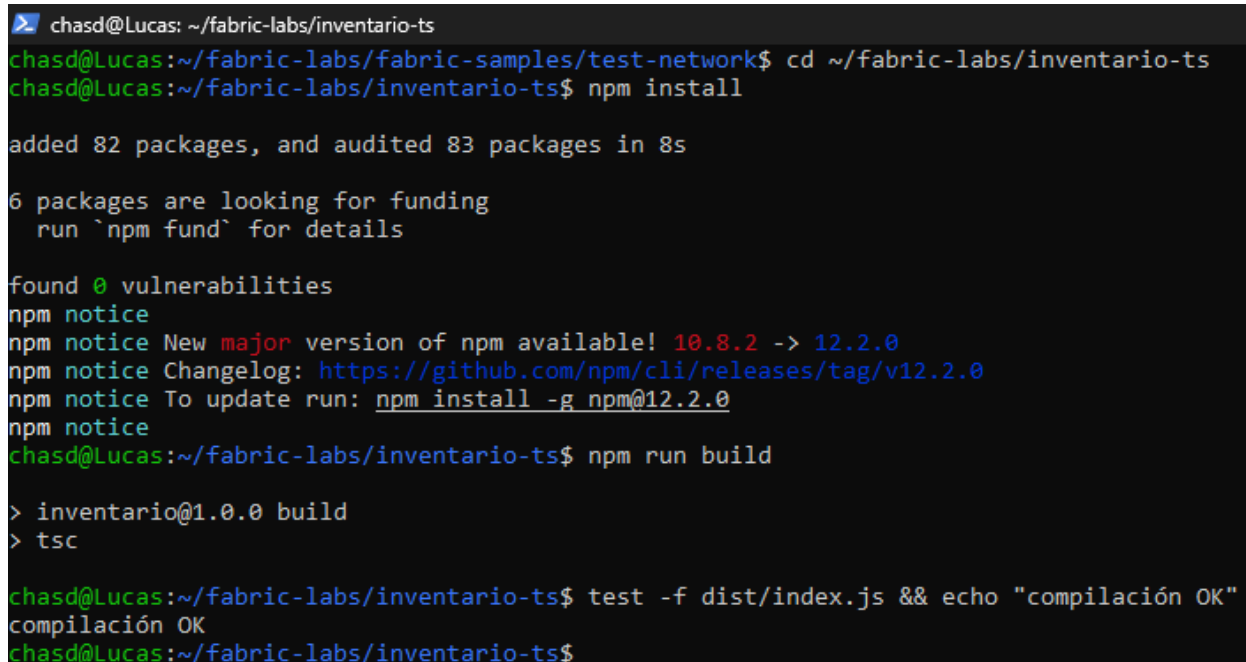
Función	Organización	Estado de origen	Estado resultante
RechazarPrestamo	Org2 (autoridad)	solicitado	disponible
RegistrarDevolucion	Org2 (autoridad)	prestado	disponible

Del código se destacan tres aspectos:

1. Doble control en cada operación. Las funciones que modifican el estado verifican primero la identidad del invocante, comparando `ctx.clientIdentity.getMSPID()` con el MSP autorizado, y luego el estado actual del equipo. Si alguna de las dos condiciones no se cumple, la función lanza una excepción, los peers no endosan la propuesta y no se escribe nada en el ledger.
2. Uso de `getTxTimestamp()` en lugar de `Date.now()`. El chaincode se ejecuta por separado en cada peer endosante y todos deben producir exactamente el mismo conjunto de lectura y escritura. `Date.now()` devolvería la hora local de cada peer y generaría resultados distintos, mientras que `getTxTimestamp()` devuelve el timestamp fijado por el cliente en la propuesta, idéntico para todos los peers. De esta forma el contrato se mantiene determinista.
3. Rich query. La función `EquiposPorEstado` utiliza `ctx.stub.getQueryResult()` con un selector JSON de CouchDB (`{ selector: { docType: 'equipo', estado } }`). Este método solo está disponible cuando la base de estado es CouchDB.

4.1. Compilación

```
cd ~/fabric-labs/inventario-ts
npm install
npm run build
test -f dist/index.js && echo "compilación OK"
```



```

> chasd@Lucas: ~/fabric-labs/inventario-ts
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ cd ~/fabric-labs/inventario-ts
chasd@Lucas:~/fabric-labs/inventario-ts$ npm install

added 82 packages, and audited 83 packages in 8s

6 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 10.8.2 -> 12.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
npm notice To update run: npm install -g npm@12.2.0
npm notice
chasd@Lucas:~/fabric-labs/inventario-ts$ npm run build

> inventario@1.0.0 build
> tsc

chasd@Lucas:~/fabric-labs/inventario-ts$ test -f dist/index.js && echo "compilación OK"
compilación OK
chasd@Lucas:~/fabric-labs/inventario-ts$
```

Figura 5. Instalación de dependencias y compilación del chaincode sin errores.

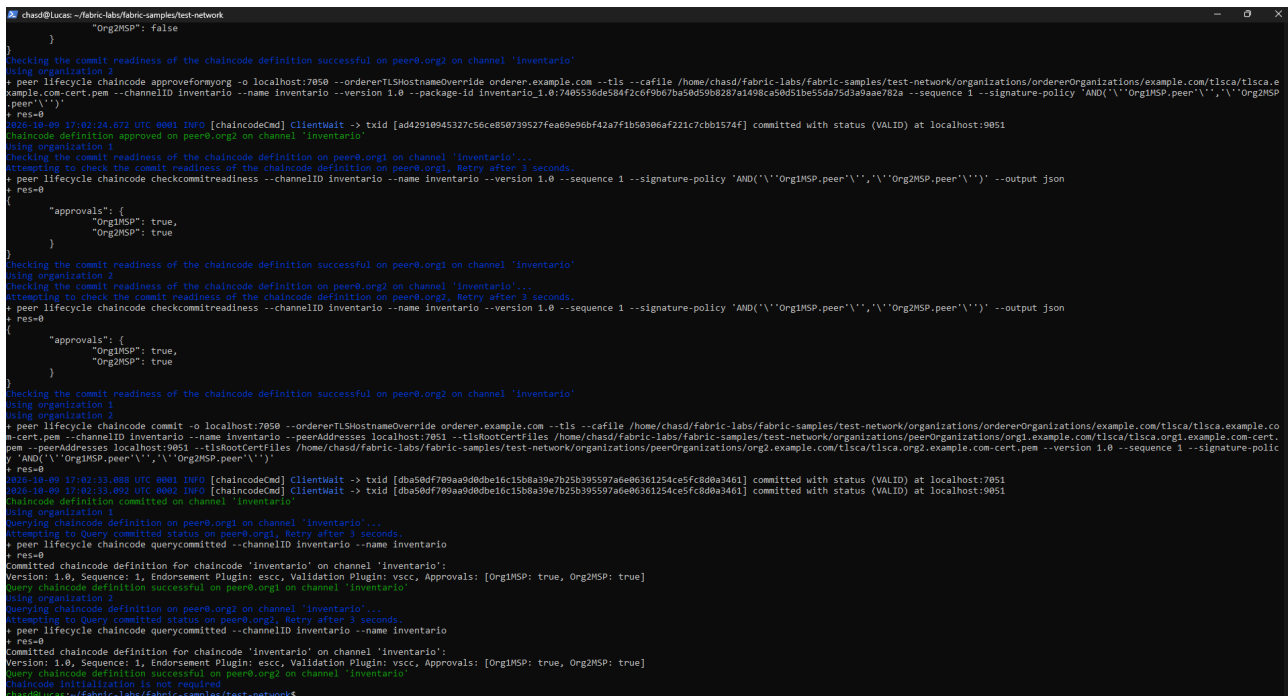
npm install instaló 82 paquetes sin vulnerabilidades. El compilador tsc no reportó errores y se generó el archivo dist/index.js, lo que confirma que los cuatro archivos fueron escritos correctamente.

5. Despliegue del chaincode

El despliegue se realizó con el script de la red de prueba, indicando el canal, el nombre, la ruta y el lenguaje del chaincode, y una política de endoso que exige la firma de un peer de cada organización:

```
cd ~/fabric-labs/fabric-samples/test-network
CHAINCODE_PATH="$HOME/fabric-labs/inventario-ts"
./network.sh deployCC -c inventario -ccn inventario -ccp "$CHAINCODE_PATH" \
  -ccl typescript -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

El script empaqueta el chaincode, lo instala en ambos peers, obtiene la aprobación de la definición por parte de cada organización y finalmente realiza el commit en el canal.



```
chad@Lucas: ~/fabric-labs/fabric-samples/test-network
}
"Org2MSP": false
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'inventario'
Using organization 2
+ peer lifecycle chaincode approvefromorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/chad/fabric-labs/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID inventario --name inventario --version 1.0 --package-id inventario_1.0:7405536de584f2c6f9b07ba50d59b8287a1498ca50d5be55da75d3a9aae782a --sequence 1 --signature-policy 'AND('Org1MSP.peer','Org2MSP.peer')'
+ res=0
002-10-09 17:02:24.672 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [ad42910945327C56ce850739527fea69e6bf42a7f1b50306af221c7cbb1574f] committed with status (VALID) at localhost:9051
Chaincode definition approved on peer0.org2 on channel 'inventario'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'inventario'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID inventario --name inventario --version 1.0 --sequence 1 --signature-policy 'AND('Org1MSP.peer','Org2MSP.peer')' --output json
+ res=0
{"approvals": {"Org1MSP": true, "Org2MSP": true}}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'inventario'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'inventario'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID inventario --name inventario --version 1.0 --sequence 1 --signature-policy 'AND('Org1MSP.peer','Org2MSP.peer')' --output json
+ res=0
{"approvals": {"Org1MSP": true, "Org2MSP": true}}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'inventario'
Using organization 1
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/chad/fabric-labs/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID inventario --name inventario --peerAddresses localhost:7051 --tlsRootCertFiles /home/chad/fabric-labs/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/chad/fabric-labs/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1.0 --sequence 1 --signature-policy 'AND('Org1MSP.peer','Org2MSP.peer')'
+ res=0
002-10-09 17:02:33.008 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [dba50df709aa908db61e1c15b8a39e7b25b395597a6e06361254ce5fc8d0a3461] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'inventario'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'inventario'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID inventario --name inventario
+ res=0
Committed chaincode definition for chaincode 'inventario' on channel 'inventario':
Version: 1.0, Sequence: 1, Endorsement Plugin: esc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'inventario'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'inventario'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID inventario --name inventario
+ res=0
Committed chaincode definition for chaincode 'inventario' on channel 'inventario':
Version: 1.0, Sequence: 1, Endorsement Plugin: esc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Chaincode initialization successful on peer0.org2 on channel 'inventario'
chad@Lucas:~/fabric-labs/fabric-samples/test-network$
```

Figura 6. Commit de la definición del chaincode con la aprobación de ambas organizaciones.

La salida Approvals: [Org1MSP: true, Org2MSP: true] confirma que las dos organizaciones aprobaron la definición antes del commit, y se verifica tanto desde peer0.org1 como desde peer0.org2. La política AND('Org1MSP.peer', 'Org2MSP.peer') implica que toda transacción deberá ser endosada por un peer de cada organización para ser considerada válida.

5.1. Configuración de identidades

Para operar alternadamente como Org1 y como Org2 se exportaron las variables comunes (binarios, configuración, TLS y certificados) y se definieron dos funciones auxiliares: usar_org, que cambia las variables CORE_PEER_* para firmar con el administrador de la organización indicada, e invocar, que envía la transacción a ambos peers tal como lo exige la política de endoso.

```

chaincode initialization is not required
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ cd ~/fabric-labs/fabric-samples/test-network
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
export CORE_PEER_TLS_ENABLED=true
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"
ORG1_CA=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
ORG2_CA=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt"

usar_org() {
  if [ "$1" = "1" ]; then
    export CORE_PEER_LOCALMSPID=Org1MSP
    export CORE_PEER_TLS_ROOTCERT_FILE=$ORG1_CA
    export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
    export CORE_PEER_ADDRESS=localhost:7051
    echo "Ahora sos Org1 (custodio)"
  else
    export CORE_PEER_LOCALMSPID=Org2MSP
    export CORE_PEER_TLS_ROOTCERT_FILE=$ORG2_CA
    export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp
    export CORE_PEER_ADDRESS=localhost:9051
    echo "Ahora sos Org2 (autoridad)"
  fi
}

invocar() {
  peer chaincode invoke -o localhost:7050 \
    --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" \
    -C inventario -n inventario \
    --peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_CA" \
    --peerAddresses localhost:9051 --tlsRootCertFiles "$ORG2_CA" \
    -c "$1"
}

chad@Lucas:~/fabric-labs/fabric-samples/test-network$

```

Figura 7. Definición de variables y funciones auxiliares `usar_org` e `invocar`.

6. Flujo entre las dos organizaciones

6.1. Inicialización del ledger y consulta por clave

```

usar_org 1
invocar '{"function":"InitLedger","Args":[]}'
sleep 3
peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'

```

```

chad@Lucas:~/fabric-labs/fabric-samples/test-network$ usar_org 1
Ahora sos Org1 (custodio)
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"InitLedger","Args":[]}'
2026-10-09 17:04:29.993 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ sleep 3
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"disponible","id":"EQ-01","solicitante":"", "ultimaAccionPor":"Org1MSP","ultimaActualizacion":"2026-10-09T17:04:29.867Z"}
chad@Lucas:~/fabric-labs/fabric-samples/test-network$

```

Figura 8. Inicialización del ledger y consulta de EQ-01.

La invocación de `InitLedger` devolvió `status:200` y dio de alta los equipos EQ-01, EQ-02 y EQ-03. La consulta por clave muestra a EQ-01 en estado disponible, con custodio Org1 y `ultimaAccionPor: Org1MSP`. Se observa que los campos aparecen ordenados alfabéticamente, ya que CouchDB devuelve el documento JSON en su propio orden y no en el orden en que fue escrito en el código.

6.2. Org1 solicita el préstamo

```

usar_org 1
invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
sleep 3
peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'

```

```

chad@Lucas:~/fabric-labs/fabric-samples/test-network$ usar_org 1
Ahora sos Org1 (custodio)
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
2026-10-09 17:07:28.233 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ sleep 3
chad@Lucas:~/fabric-labs/fabric-samples/test-network$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"solicitado","id":"EQ-01","solicitante":"Org2","ultimaAccionPor":"Org1MSP","ultimaActualizacion":"2026-10-09T17:07:28.196Z"}
chad@Lucas:~/fabric-labs/fabric-samples/test-network$

```

Figura 9. Solicitud de préstamo realizada por Org1.

La transacción fue aceptada y, luego del commit, EQ-01 quedó en estado solicitado con solicitante: Org2. A partir de este momento el turno corresponde a Org2.

6.3. Bloqueo de Org1

Con la misma identidad de Org1 se intentó volver a solicitar el equipo y, a continuación, aprobar la propia solicitud:

```
usar_org 1
invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'

chasd@Lucas: ~/fabric-labs/fabric-samples/test-network
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ usar_org 1
Ahora sos Org1 (custodio)
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
Error: endorsement failure during invoke. response: status:500 message:"no se puede solicitar EQ-01: estado actual \"solicitado\". El turno es de Org2MSP"
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'
Error: endorsement failure during invoke. response: status:500 message:"solo la autoridad (Org2MSP) puede aprobar; cliente recibido: Org1MSP"
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$
```

Figura 10. Rechazo de las dos operaciones intentadas por Org1.

Ambas invocaciones fallaron con status:500 en la fase de endoso, cada una por un motivo diferente:

- Error por estado: no se puede solicitar EQ-01: estado actual "solicitado". El turno es de Org2MSP. Org1 tiene el rol necesario para solicitar, pero la operación exige que el equipo esté disponible, y la máquina de estados no permite repetir la transición.
- Error por rol: solo la autoridad (Org2MSP) puede aprobar; cliente recibido: Org1MSP. La aprobación está reservada a Org2, independientemente del estado del equipo.

En los dos casos el chaincode lanza la excepción durante la simulación, los peers no firman la propuesta y la transacción nunca llega al orderer, por lo que el ledger permanece sin cambios.

6.4. Org2 aprueba el préstamo

```
usar_org 2
invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'
sleep 3
peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
```

```
chasd@Lucas: ~/fabric-labs/fabric-samples/test-network
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ usar_org 2
Ahora sos Org2 (autoridad)
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'
2026-10-09 17:11:31.711 UTC [Org1:0] INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ sleep 3
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"prestado","id":"EQ-01","solicitante":"Org2","ultimaAccionPor":"Org2MSP","ultimaActualizacion":"2026-10-09T17:11:31.712Z"}
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$
```

Figura 11. Aprobación del préstamo por parte de Org2.

Al operar con la identidad de Org2, la aprobación fue aceptada. EQ-01 pasó a estado prestado y el campo ultimaAccionPor registra ahora a Org2MSP.

6.5. Org2 registra la devolución

```
usar_org 2
invocar '{"function":"RegistrarDevolucion","Args":["EQ-01"]}'
sleep 3
peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
```

```
chasd@Lucas: ~/fabric-labs/fabric-samples/test-network
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ usar_org 2
Ahora sos Org2 (autoridad)
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ invocar '{"function":"RegistrarDevolucion","Args":["EQ-01"]}'
2026-10-09 17:12:07.420 UTC [Org1:0] INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ sleep 3
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"disponible","id":"EQ-01","solicitante":"","ultimaAccionPor":"Org2MSP","ultimaActualizacion":"2026-10-09T17:12:07.731Z"}
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$
```

Figura 12. Registro de la devolución de EQ-01.

La devolución cerró el ciclo: EQ-01 volvió a estado disponible y el campo solicitante quedó vacío. El último cambio quedó atribuido a Org2.

7. Rich query sobre CouchDB

Se consultaron todos los equipos en estado disponible mediante la función EquiposPorEstado, antes y después de una nueva solicitud de Org1.

```
peer chaincode query -C inventario -n inventario -c '{"Args":
["EquiposPorEstado","disponible"]}'
```

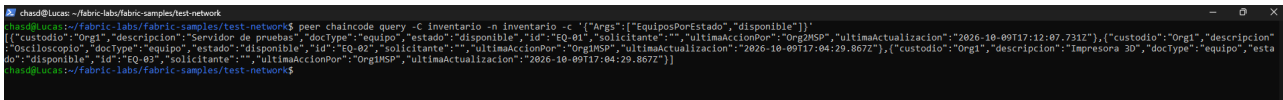


Figura 13. Rich query antes de la nueva solicitud: tres equipos disponibles.

```
usar_org 1
invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
sleep 3
peer chaincode query -C inventario -n inventario -c '{"Args":
["EquiposPorEstado","disponible"]}'
```

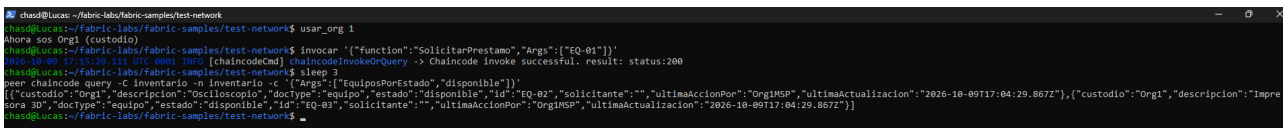


Figura 14. Rich query después de la nueva solicitud: solo EQ-02 y EQ-03.

En la primera consulta se obtuvieron los tres equipos. Luego de la nueva solicitud, la misma consulta devolvió únicamente EQ-02 y EQ-03. El resultado cambió porque la rich query no busca por clave sino por el contenido de los documentos: CouchDB devuelve los que cumplen el selector `docType = "equipo"` y `estado = "disponible"`, y al confirmarse la solicitud el documento de EQ-01 pasó a tener `estado = "solicitado"`.

Con LevelDB esta consulta no sería posible de forma directa, ya que esa base solo almacena pares clave-valor sin interpretar el contenido. Habría que recorrer todas las claves con `getStateByRange` y filtrar dentro del chaincode, o bien mantener manualmente un índice mediante claves compuestas (por ejemplo `estado~id`) y actualizarlo en cada cambio de estado.

Como verificación adicional se accedió a la interfaz web de CouchDB (Fauxton) en http://localhost:5984/_utils, correspondiente a `couchdb0`, y se abrió el documento EQ-01 de la base `inventario_inventario`, cuyo nombre se forma con el canal y el chaincode:

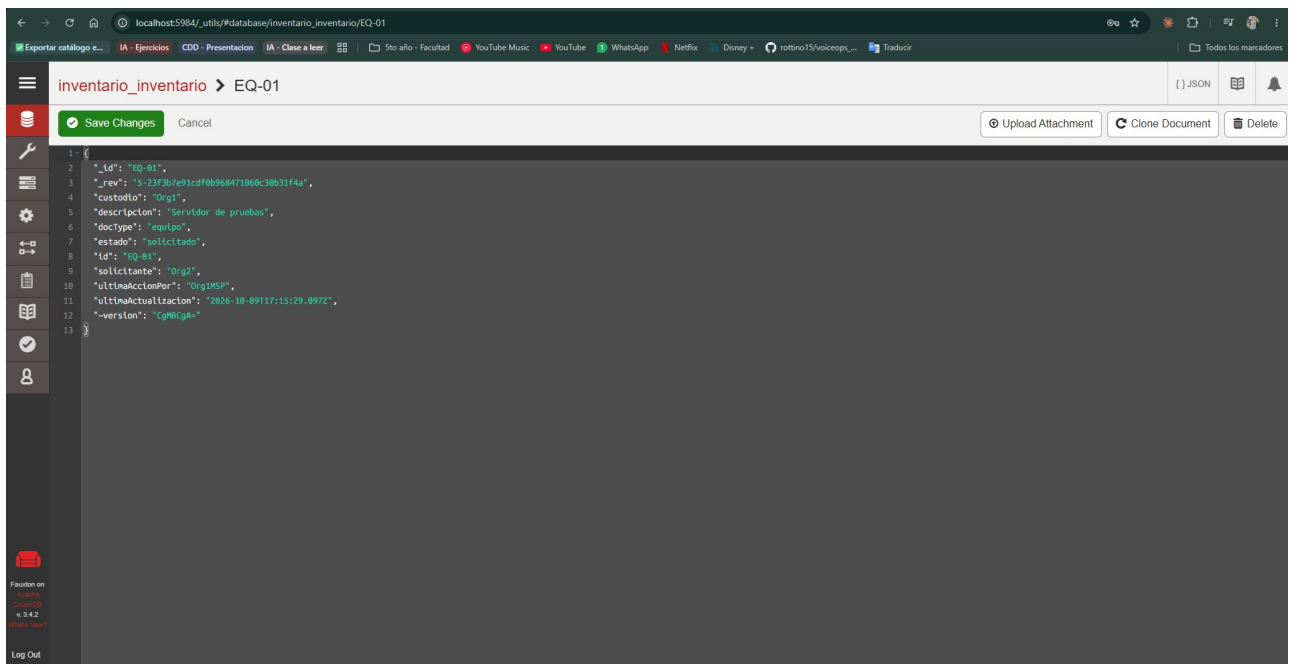


Figura 15. Documento EQ-01 almacenado en CouchDB (couchdb0), visto desde Fauxton.

El documento refleja el estado actual del activo (solicitado). El campo `_rev` comienza con 5-, lo que indica que CouchDB registró cinco revisiones del documento, en coincidencia con las cinco transacciones válidas que lo modificaron. Cabe aclarar que CouchDB solo conserva el valor vigente; el historial completo con sus transacciones se obtiene del ledger, como se muestra en el apartado siguiente. Por otra parte, editar el documento directamente desde Fauxton modificaría solo la copia local de un peer sin pasar por el endoso ni el consenso, por lo que no se realizó ninguna modificación.

8. Historial del activo

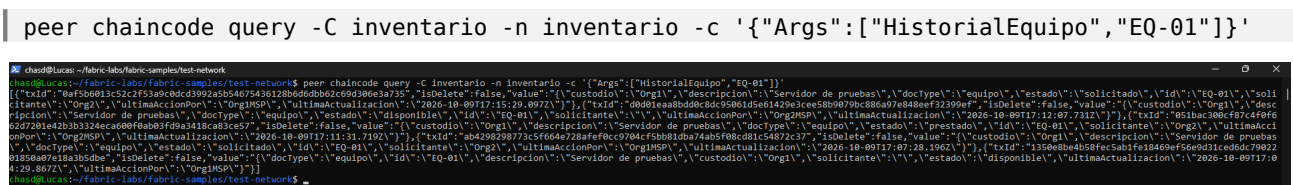


Figura 16. Historial completo de EQ-01 obtenido con `getHistoryForKey`.

La consulta devolvió cinco entradas, ordenadas de la más reciente a la más antigua. Cada una tiene un identificador de transacción distinto, el valor completo del activo en ese momento y la organización que realizó el cambio:

N.º	txId (inicio)	Estado	Realizado por	Operación
5	0af5b601...	solicitado	Org1MSP	Nueva solicitud (17:15:29)
4	d0d01eaa...	disponible	Org2MSP	Devolución (17:12:07)
3	051bac30...	prestado	Org2MSP	Aprobación (17:11:31)
2	ab429829...	solicitado	Org1MSP	Solicitud (17:07:28)
1	1350e8be...	disponible	Org1MSP	Alta mediante InitLedger (17:04:29)

Los dos intentos rechazados de Org1 no figuran en el historial, ya que no superaron el endoso y nunca se escribieron. También se observa que la primera entrada conserva los

campos en el orden del código, mientras que las siguientes aparecen en orden alfabético: esto se debe a que las demás operaciones leyeron el documento desde CouchDB, lo modificaron y lo volvieron a escribir.

Este historial no proviene del World State, que solo guarda el valor actual, sino del transaction log, es decir, de la cadena de bloques. No puede falsificarse porque cada bloque incluye el hash del anterior, cada transacción está firmada por el cliente y por los peers endosantes de ambas organizaciones, y cada organización conserva su propia copia del ledger. La autoría de cada cambio queda registrada tanto en el campo `ultimaAccionPor` del activo como, de manera criptográfica, en la identidad del creador incluida en cada transacción.

9. Baja de la red

```
cd ~/fabric-labs/fabric-samples/test-network
./network.sh down
docker ps --format "{{.Names}}" | grep -E "peer|orderer|couchdb|inventario" || echo "red detenida OK"
```

```
chasd@Lucas: ~/fabric-labs/fabric-samples/test-network
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ ./network.sh down
Stopping network
[+] down 9/9
✓ Container peer0.org1.example.com Removed
✓ Container orderer.example.com Removed
✓ Container peer0.org2.example.com Removed
✓ Container couchdb0 Removed
✓ Container couchdb1 Removed
✓ Volume compose_orderer.example.com Removed
✓ Network fabric_test Removed
✓ Volume compose_peer0.org1.example.com Removed
✓ Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-inventario_1.0-7405536de584f2c6f9b67ba50d59b8287a1498ca50d51be55da75d3a9aae782a-0c0dc64cd4cfd0ef9f758e4485908f3b53b0329c43a144c91d74fd968b3451:latest
Deleted: sha256:8a1a7051258409f99845bbe30269349258897a5774e2bc1fdb908983784c0254
Untagged: dev-peer0.org1.example.com-inventario_1.0-7405536de584f2c6f9b67ba50d59b8287a1498ca50d51be55da75d3a9aae782a-50eb3d733ddfb4303d4f2b7052c40c8ba5923cd1922b01058f92398d24c4ce5d:latest
Deleted: sha256:f9311fc864c6cfab8a686271881a795898d8a70ca0aad5042d07587818d75dc
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$ docker ps --format "{{.Names}}" | grep -E "peer|orderer|couchdb|inventario" || echo "red detenida OK"
red detenida OK
chasd@Lucas:~/fabric-labs/fabric-samples/test-network$
```

Figura 17. Baja de la red y verificación de que no quedan contenedores de Fabric.

El script eliminó los contenedores de los peers, el orderer y las dos instancias de CouchDB, junto con los volúmenes, la red `fabric_test` y las imágenes generadas para el chaincode. Los mensajes `no such volume` corresponden a nombres de volúmenes de versiones anteriores del script que no existen en esta instalación y no constituyen un error. La verificación final devolvió `red detenida OK`; los demás contenedores del equipo no se vieron afectados.

10. Análisis de las capas de confianza

A partir de lo observado durante la práctica se identifican cinco capas que, en conjunto, permiten que las organizaciones confíen en el registro sin depender de un intermediario:

Capa	Qué garantiza	Dónde se observó en el trabajo
Identidad (MSP y certificados X.509)	Cada participante está identificado y autorizado; la red es permitida.	Cambio de identidad con <code>userOrg</code> y lectura del MSP con <code>getMSPID()</code> dentro del contrato.
Lógica del chaincode	Las reglas de negocio se aplican igual para todos los participantes.	Controles por rol y por estado que bloquearon a <code>Org1</code> (apartado 6.3).
Política de endoso	Ninguna organización puede escribir por sí sola.	Política <code>AND(Org1, Org2)</code> , aprobación de ambas en el despliegue e

Capa	Qué garantiza	Dónde se observó en el trabajo
		invocaciones dirigidas a los dos peers.
Ordenamiento y validación	Orden único de las transacciones y validación de firmas y versiones antes del commit.	Orderer Raft; el nuevo estado es visible recién luego del commit, de ahí la espera entre invocación y consulta.
Inmutabilidad e historial	Lo registrado no puede alterarse y queda disponible para auditoría.	Historial de EQ-01 con cinco transacciones y su autor (apartado 8).

11. Problemas encontrados

1. Uso de PowerShell. La primera verificación del entorno se realizó desde Windows PowerShell. Como los scripts de Fabric requieren Bash, se pasó a trabajar dentro de la distribución Ubuntu de WSL2, donde fue necesario instalar git, curl, jq y Node.js, ya que las versiones instaladas en Windows no están disponibles dentro de Linux.
2. Instalación de Ubuntu. Al ejecutar `wsl --install -d Ubuntu` se obtuvo el error `ERROR_ALREADY_EXISTS`. El listado `wsl -l -v` mostró que Ubuntu ya estaba instalado y en ejecución, por lo que no se requirió ninguna acción adicional.
3. Conexión de VS Code a la distribución incorrecta. Al conectar VS Code a WSL se abrió la distribución `docker-desktop`, que es la que utiliza internamente Docker y estaba configurada como predeterminada. Esa distribución está basada en Alpine y no permite ejecutar el servidor de VS Code (error VS Code Server for WSL closed unexpectedly, por falta de `libstdc++`). Se resolvió estableciendo Ubuntu como distribución predeterminada con `wsl --set-default Ubuntu` y abriendo nuevamente el proyecto con `code ~/fabric-labs/inventario-ts` desde la terminal de Ubuntu.
4. Latencia entre invocación y consulta. Una invocación devuelve `status:200` en cuanto la propuesta es endosada, pero el nuevo estado es visible recién después del commit del bloque. Por ese motivo se incluyó una espera de tres segundos (`sleep 3`) antes de cada consulta.

12. Respuestas finales

12.1. Controles que impidieron actuar a Org1

Intervinieron dos controles distintos. El control por estado impidió que Org1 volviera a solicitar el equipo: tenía el rol correcto, pero la operación exige el estado disponible y el equipo se encontraba solicitado. Este control depende del dato almacenado y deja de bloquear cuando Org2 resuelve la solicitud. El control por rol impidió que Org1 aprobara su propia solicitud, ya que la aprobación exige el MSP Org2MSP. Este control depende de la identidad del firmante y se mantiene en cualquier estado. El primero responde a si la operación corresponde en ese momento del proceso; el segundo, a si quien la ejecuta está autorizado. Combinados, evitan que una única organización recorra todo el flujo por su cuenta.

12.2. Consulta por clave frente a rich query

ReadEquipo utiliza `getState(id)` y recupera un único valor a partir de su clave exacta; funciona tanto con LevelDB como con CouchDB. EquiposPorEstado utiliza `getQueryResult(selector)` y busca por el contenido de los valores, pudiendo devolver varios resultados sin conocer sus claves. Esta segunda consulta requiere CouchDB, porque es la única base de estado que almacena los valores como documentos JSON y admite consultas mediante selectores. LevelDB trata los valores como datos opacos y solo permite búsquedas por clave o por rango de claves.

12.3. Uso de `getTxTimestamp()`

Cada peer endosante ejecuta el chaincode de forma independiente y, para que la transacción sea válida, todos deben obtener exactamente el mismo resultado. Si se usara `Date.now()`, cada peer registraría su propia hora local, los conjuntos de escritura diferirían y la transacción sería rechazada por inconsistencia en los endosos. `getTxTimestamp()` devuelve el timestamp que el cliente incluyó en la propuesta, que es el mismo para todos los peers, y garantiza así un comportamiento determinista.

12.4. Origen y confiabilidad del historial

El historial proviene del transaction log, es decir, de los bloques del ledger, y no del World State, que solo conserva el valor vigente. Es confiable porque los bloques están encadenados mediante hashes, de modo que cualquier alteración resulta detectable; porque cada transacción está firmada por su creador y por los endosantes de ambas organizaciones; y porque cada organización mantiene una copia propia del ledger. La organización responsable de cada cambio queda registrada en el campo `ultimaAccionPor` de cada versión del activo y, en forma criptográfica, en el certificado del creador que acompaña a cada transacción.

12.5. Capas de confianza

Las cinco capas son la identidad provista por el MSP, la lógica del chaincode, la política de endoso, el ordenamiento y la validación de las transacciones, y la inmutabilidad del ledger con su historial. En este trabajo se observaron, respectivamente, en el cambio de identidad entre Org1 y Org2, en los controles que bloquearon a Org1, en la aprobación conjunta del despliegue y la exigencia de dos endosos, en la espera necesaria hasta el commit de cada bloque y en el historial de EQ-01 con sus cinco transacciones (véase el apartado 10).

12.6. Caso C: certificados académicos

Para un registro de certificados académicos, el activo Certificado podría contener los siguientes campos: `docType`, `id` (número de certificado), datos del egresado (legajo o DNI y nombre), `carrera`, `titulo`, `fechaEgreso`, `hashDocumento` (huella SHA-256 del diploma, sin almacenar el archivo en la cadena), `emisor`, `estado`, `motivo` (para rechazos o revocaciones), `ultimaActualizacion` y `ultimaAccionPor`.

Los estados posibles serían solicitado, emitido, rechazado y revocado, con las siguientes transiciones y roles:

Operación	Organización	Transición
SolicitarEmision	Org1: Facultad	nuevo → solicitado

Operación	Organización	Transición
EmitirCertificado	Org2: Rectorado o Ministerio	solicitado → emitido
RechazarEmision	Org2: Rectorado o Ministerio	solicitado → rechazado
RevocarCertificado	Org2: Rectorado o Ministerio	emitido → revocado

La Facultad cargaría la solicitud con los datos académicos y el hash del diploma, sin poder emitir ni revocar. El Rectorado o el Ministerio validaría la información y tendría a su cargo la emisión, el rechazo y la revocación, sin poder crear solicitudes. Terceros como empresas u otras universidades podrían participar solo con permisos de lectura para verificar la autenticidad de un certificado y su vigencia. Se mantendría una política de endoso $AND(Org1, Org2)$ y se agregarían rich queries como `CertificadosPorEstado` o `CertificadosPorAlumno` sobre CouchDB.

13. Lista de verificación

Requisito	Estado
Red levantada con <code>-s couchdb</code>	Cumplido
Contenedores <code>couchdb0</code> y <code>couchdb1</code> visibles en <code>docker ps</code>	Cumplido
Cuatro archivos del <code>chaincode</code> creados manualmente	Cumplido
Canal inventario creado	Cumplido
<code>npm run build</code> generó <code>dist/index.js</code>	Cumplido
Deploy con <code>Approvals: [Org1MSP: true, Org2MSP: true]</code>	Cumplido
Solicitud de Org1 (estado solicitado)	Cumplido
Org1 no puede volver a solicitar (error por estado)	Cumplido
Org1 no puede aprobar (error por rol)	Cumplido
Aprobación de Org2 (estado prestado) y devolución	Cumplido
Rich query antes y después de una solicitud	Cumplido
Historial con varios <code>txId</code>	Cumplido
Red detenida con <code>./network.sh down</code>	Cumplido